🏡 AI House Price Predictor: Full-Stack Project Guide

Welcome to theAI HousePricePredictor project! In thistutorial,youwilllearnhowto connect a Machine Learning model to a modern web application.

By the end of this guide, you will have three separate pieces of technology talking to each other:

1. **Machine Learning (Python/Scikit-Learn):** The brain that predicts the price.
2. **Backend API (Python/FastAPI):** The server that listens for requests and asks the brain for an answer.
3. **Frontend UI (React/Vite):** The user interface where users enter data and see the results.

🧠 Project Architecture: How it works

1. The **User** types house details (Square Footage, Rooms, etc.) into the **React Frontend**.
2. React packages this data into a JSON object and sends it via an HTTP POST request to the **FastAPI Backend**.
3. The Backend receives the data, formats it, and feeds it into the pre-trained **Scikit-Learn Model** (`.joblib` file).
4. The Model calculates the predicted price.
5. FastAPI sends that price back to React.
6. React updates the screen to show the user the estimated value.

🛠️ Prerequisites

Before you begin, ensure you have the following installed on your computer:

- **Python (3.8 or newer):** For the ML model and Backend.
- **Node.js (16 or newer):** For running the React Frontend.
- **A Code Editor:** VS Code is highly recommended.

Note: You will need to open two separate terminal windows during this process—one for the backend, and one for the frontend.

🚀 Step-by-Step Execution Guide

Part 1: Train the Machine Learning Model

First, we need to train our AI and save its "brain" to a file so our API can use it.

1. Open your terminal and navigate to the `ML` folder:
   `cd ML`
2. (Optional but recommended) Create and activate a Python virtual environment:
   ```
   python -m venv venv
   # On Windows: venv\Scripts\activate
   # On Mac/Linux: source venv/bin/activate
   ```
3. Install the required Python libraries for Machine Learning:
   `pip install pandas numpy scikit-learn joblib`
4. Run the training script:
   `python HousePricePredictor.py`
5. **Success Check:** You should see a message saying the model was trained, and a new file named `house_price_model.joblib` will appear in your ML folder.

Part 2: Start the FastAPI Backend

Now we will start the server that hosts our trained model.

1. Open a **NEW** terminal window and navigate to the `FastAPI` folder:
   `cd FastAPI`
2. If you created a virtual environment in Part 1, activate it here as well.
3. Install the required web server libraries:
   `pip install fastapi uvicorn pydantic`
4. Start the FastAPI server:
   `uvicorn main:app --reload`
5. **Success Check:** You should see a message like `Uvicorn running on http://127.0.0.1:8000`.
   - **Pro Tip for Students:** Open `http://127.0.0.1:8000/docs` in your browser. FastAPI automatically generates a beautiful testing interface (Swagger UI) where you can test your prediction API without even needing the frontend!

Part 3: Start the React Frontend

Finally, let's start the beautiful user interface.

1. Open a **THIRD** terminal window and navigate to the `React/frontend` folder:
   `cd React/frontend`

2. Install the Node.js dependencies (this reads the `package.json` file and downloads libraries like React, Framer Motion, and Lucide Icons):
   `npm install`
3. Start the Vite development server:
   `npm run dev`
4. **Success Check:** Your terminal will show a local URL, usually `http://localhost:5173`. Click it (or copy-paste it into your browser) to view your app!

🎓 Connecting the Dots (For Students)

As you play with the app, take a look at the code to see how they connect:

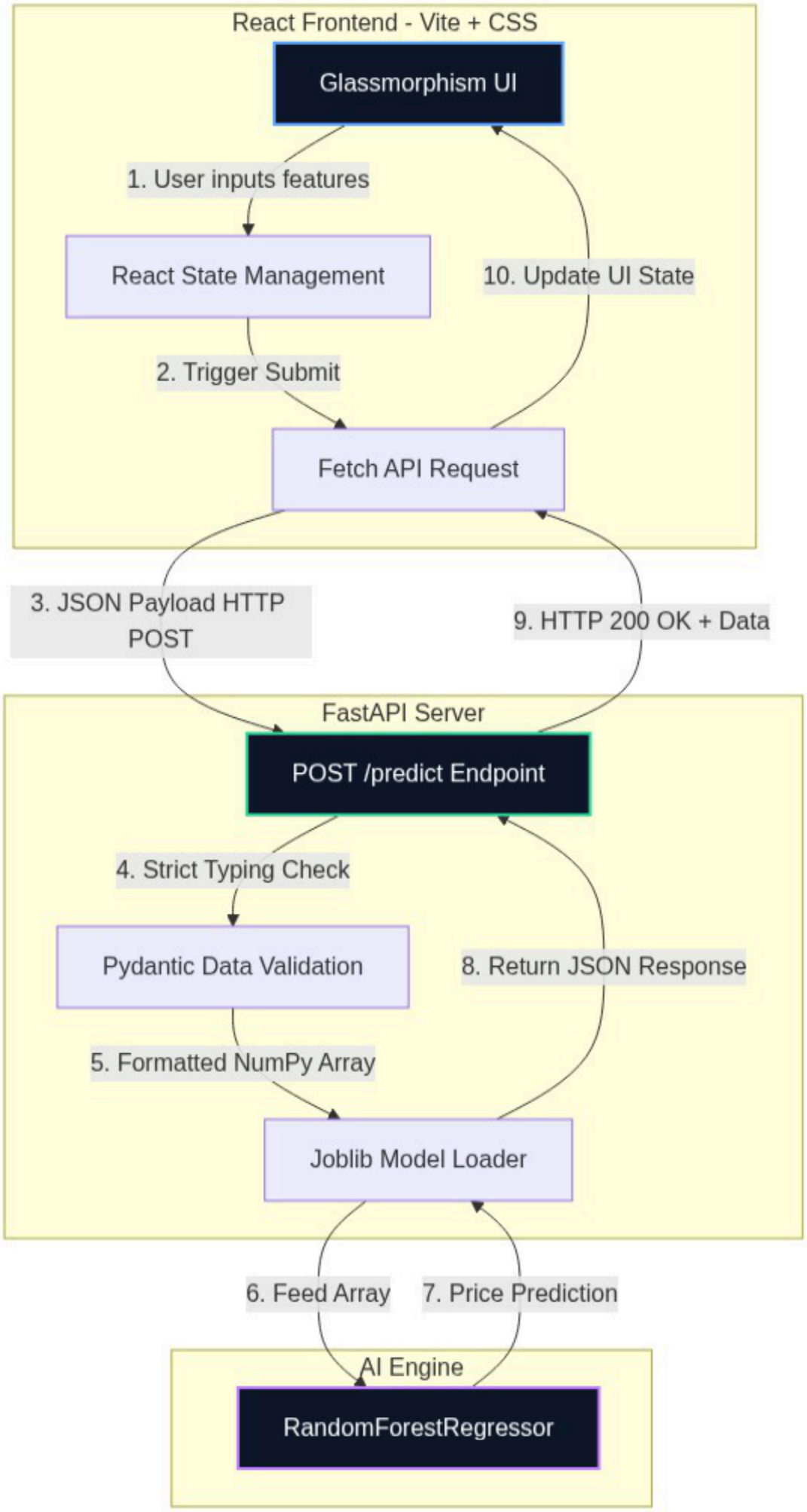
- **In FastAPI/main.py:** Look for the `CORSMiddleware` section. This is a security feature. Because our React app runs on port `5173` and our Backend runs on port `8000`, the browser normally blocks them from talking. We explicitly tell FastAPI to allow `connections from http://localhost:5173.`
- **In `React/frontend/src/App.jsx`:** Look for the `fetch('http://localhost:8000/predict', ...)` line inside the `handlePredict` function. This is the exact moment the frontend knocks on the backend's door to ask for a prediction.

🛑 Common Troubleshooting

- **"Failed to fetch" Error in React:** This almost always means your FastAPI server isn't running, or it's running on a different port than 8000. Ensure Part 2 is running in a terminal in the background.
- **"Model not loaded" Error:** Make sure you ran the `HousePricePredictor.py` script first so the `.joblib file` exists!

Happy Coding! Feel free to modify the CSS or the Machine Learning logic to make this project your own.

www.Vi4Academy.com


React Frontend - Vite + CSS
Glassmorphism UI
1. User inputs features
React State Management
10. Update UI State
2. Trigger Submit
Fetch API Request
3. JSON Payload HTTP POST
9. HTTP 200 OK + Data
FastAPI Server
POST /predict Endpoint
4. Strict Typing Check
Pydantic Data Validation
8. Return JSON Response
5. Formatted NumPy Array
Joblib Model Loader
6. Feed Array
7. Price Prediction
AI Engine
RandomForestRegressor

Core Functionalities by Component

1. The Machine Learning Engine (Scikit-Learn)

- **Data Synthesis & Training:** The `train.py` script acts as the data science sandbox. It generates a synthetic dataset representing real estate data and trains a `RandomForestRegressor`.
- **Pattern Recognition:** The Random Forest algorithm builds multiple decision trees during training and averages their outputs to predict the house price based on non-linear relationships between square footage, rooms, age, and location.
- **Serialization:** Using `joblib`, the fully trained model is serialized (frozen in its trained state) and saved as a `.joblib` file so it can be deployed without needing to retrain the model every time the server starts.

2. The Backend Server (FastAPI)

- **Model Serving:** Upon startup, FastAPI immediately loads the `.joblib` file into memory. If the model is missing, it catches the error gracefully rather than crashing. **Data**
- **Validation:** Using **Pydantic** (`HouseFeatures`), the backend enforces strict data types. If a student tries to send a string like "three" instead of the integer 3 for `rooms`, FastAPI automatically rejects the request with a helpful 422 error, protecting the ML model from bad data.
- **Cross-Origin Resource Sharing (CORS):** Because the React app runs on port `5173` and FastAPI runs on port `8000`, browsers will block communication by default. The `CORSMiddleware` explicitly authorizes the React app to communicate with the Python server.
- **Transformation:** It translates the incoming JSON dictionary into a 2D NumPy array, which is the exact mathematical format `scikit-learn` requires for prediction.

3. The Frontend Application (React + Vite)

- **State Management:** React uses the `useState` hook to keep track of the user's input continuously. It maintains a single source of truth (`formData`) that represents the exact state of the UI at any given millisecond.
- **Asynchronous Communication:** The `handlePredict` function utilizes modern asynchronous JavaScript (`async/await` + `fetch`) to send the POST request to the backend without freezing the browser page.
- **Interactive Feedback:** While waiting for the AI to process, the UI dynamically disables the submit button and changes the text to "Processing Tensor...", giving the user immediate visual feedback. Once the backend responds, the state updates and the UI conditionally renders the formatted price result.

The Request-Response Lifecycle (How it all connects)

When teaching this, you can trace a single request from start to finish:

1. **Input:** A user types `1500` into the Square Footage box. React updates `formData.sqrft` in memory.
2. **Dispatch:** The user clicks "Predict". React intercepts the form submission, prevents the page from reloading, and packages `formData` into a JSON string.
3. **Transit:** The browser sends an HTTP POST request to `http://localhost:8000/predict`.
4. **Validation:** FastAPI receives the JSON, checks that all four required fields exist and are numbers, and passes them to the endpoint logic.
5. **Inference:** Python converts the numbers into an array, hands them to the loaded Random Forest model, and asks for a prediction.
6. **Return:** Python formats the resulting float (e.g., `345000.52`) into JSON and sends it back to the browser.
7. **Render:** React receives the JSON, extracts the price, updates the `prediction` state, and the browser repaints the screen to show the final green result card.

---

www.Vi4Academy.com